Practical – 1(A)

Aim : Import the data warehouse data in Microsoft Excel and create the Pivot table and Pivot Chart.

Pivot Table : A Pivot Table is a powerful Excel tool used to summarize, analyze, and organize large amounts of data**.**

Pivot Chart : A Pivot Table is a powerful Excel tool used to summarize, analyze, and organize large amounts of data.

Steps To Create a **Pivot Table and Pivot Chart :**

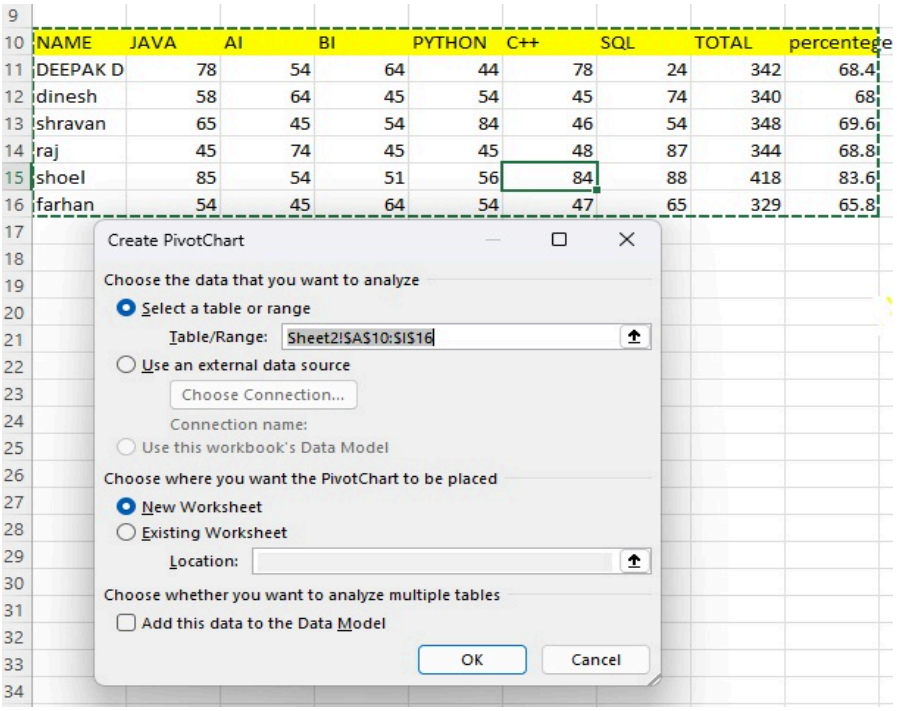
Step 1: Import Data into Excel

1. Open Microsoft Excel and Create a new worksheet.
2. Enter the above data in rows and columns and Save the file .

NAME	JAVA	AI	BI	PYTHON	C++	SQL	TOTAL	percentege
DEEPAK D	78	54	64	44	78	24	342	68.4
dinesh	58	64	45	54	45	74	340	68
shravan	65	45	54	84	46	54	348	69.6
raj	45	74	45	45	48	87	344	68.8
shoel	85	54	51	56	84	88	418	83.6
farhan	54	45	64	54	47	65	329	65.8

Step 2: Create Pivot Table

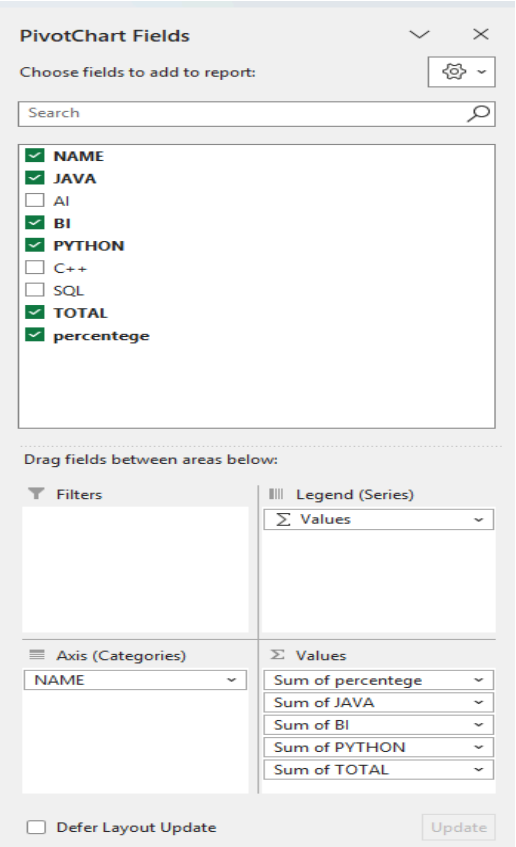
1. **Select the complete data and Click Insert → PivotTable .**
2. **Choose From Table/Range.**
3. **Select New Worksheet and Click OK .**



NAME	JAVA	AI	BI	PYTHON	C++	SQL	TOTAL	percentege
DEEPAK D	78	54	64	44	78	24	342	68.4
dinesh	58	64	45	54	45	74	340	68
shravan	65	45	54	84	46	54	348	69.6
raj	45	74	45	45	48	87	344	68.8
shoel	85	54	51	56	84	88	418	83.6
farhan	54	45	64	54	47	65	329	65.8

Step 3: Design Pivot Table

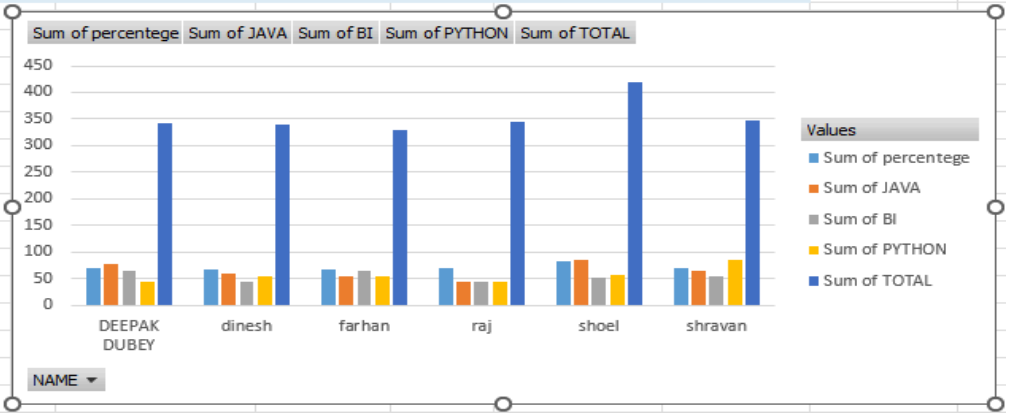
- Drag Product → Rows
- Drag Region → Columns
- Drag Sales → Values (Sum of Sales)

- 

Step 4: Create Pivot Chart

1. Click anywhere on the Pivot Table
2. Go to Insert → PivotChart
3. Click OK

Row Labels	Sum of percentege	Sum of JAVA	Sum of BI	Sum of PYTHON	Sum of TOTAL
DEEPAK DUBEY	68.4	78	64	44	342
dinesh	68	58	45	54	340
farhan	65.8	54	64	54	329
raj	68.8	45	45	45	344
shoel	83.6	85	51	56	418
shravan	69.6	65	54	84	348
Grand Total	**424.2**	**385**	**323**	**337**	**2121**



Practical - 1(B)

Aim : Import the cube in Microsoft Excel and Create the Pivot table and Pivot Chart to perform data analysis.

Pivot Table : A Pivot Table is a powerful Excel tool used to summarize, analyze, and organize large amounts of data.

Pivot Chart : A Pivot Table is a powerful Excel tool used to summarize, analyze, and organize large amounts of data.

Steps To Create a **Pivot Table and Pivot Chart :**

Step 1: Import Data into Excel

1. Open Microsoft Excel and Create a new worksheet.
2. Enter the above data in rows and columns and Save the file .

1	City	City	City	City	Total Profits in each city
2	Oklahoma	$62.15		Lagos	0.00
3	Wollongo	-$288.77		Istanbul	0.00
4	Brisbane	$919.97		Tegucigalp	0.00
5	Berlin	-$96.54		Philadelpl	0.00
6	Dakar	$311.52		Lahore	0.00
7	Sydney	$763.28		Stockholm	0.00
8	Porirua	$564.84		Manila	0.00
9	Hamilton	$996.48		Kano	0.00
10	Oklahoma	$54.71		Hanover	0.00
11					
12	City	City	City	City	Total Profits in each city
13	Seattle	$5.48		Toulouse	0.00
14	Seattle	$4.66		Houston	0.00
15	Kabul	$647.55		Ankara	0.00
16	Jizan	$1,151.40		Buenos Ai	0.00
17	Toledo	$622.02		Izmir	0.00
18	Mudanjiar	$1,036.08		San Anton	0.00
19	Paris	$186.95		Lancaster	0.00
20	Arlington	-$2.52		Bangkok	0.00
21	Prato	$3,979.08		Panama C	0.00
22	Townsvill	$28.40		San Pedro	0.00
23					
24	City	City	City	City	Total Profits in each city
25	Uvinza	$818.28		Chicago	0.00
26	Bytom	$276.84		Bornova	0.00
27	Seattle	$4.86		San Migue	0.00
28	Suzhou	$358.02		Birmingha	0.00
29	Edinburgh	$1,898.40		Canberra	0.00
30	Juárez	$445.52		Salvador	0.00
31	Soyapang	$526.50		Manukau	0.00
32	Taipei	$720.36		Port Harcc	0.00
33	Leipzig	$1,364.24		Seoul	0.00
34	Troy	$121.44		Amsterda	0.00
35					

Step 2: Create Cube from Pivot Chart Table

Data Cube : A Data Cube is a multi-dimensional data structure used to store and analyze large amounts of data from different perspectives such as time, region, product, or category. it allow fast and efficient data analysis.

Pivot Table and Cube Relationship : A Pivot Table can be connected to a data cube(OLAP Cube) to perform advanced analysis. The cube provides structured, summarized data, while the Pivot Table and Pivot Chart are used to explore and visualize it.

Step-to : Create Cube from Pivot Chart Table

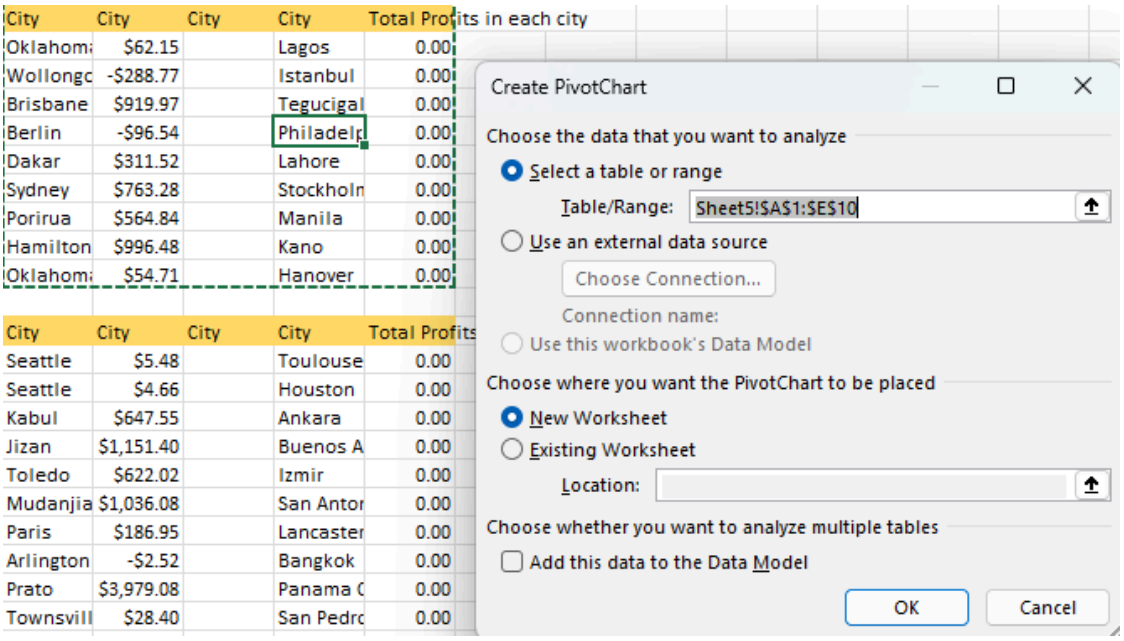
1. Copy data from Pivot Chart Table

2. Paste selected/copy data in row column with picture link

3. Go to Picture Format → Select Picture Effect →Select 3D Rotation → Select faces of Cube

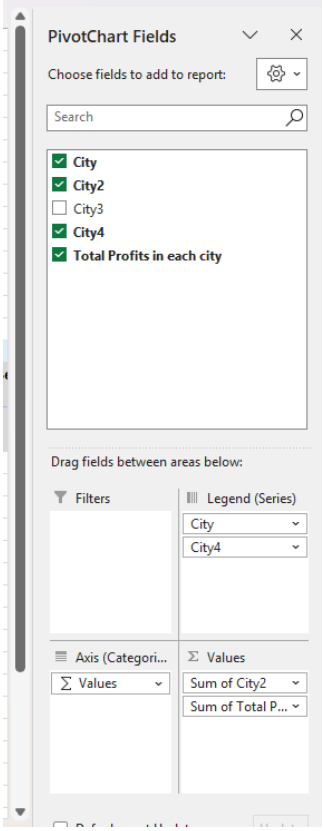
Step 3: Create Pivot Table

1. Select the complete data and Click Insert → PivotTable .
2. Choose From Table/Range.
3. Select New Worksheet and Click OK .



Step 4: Design Pivot Table

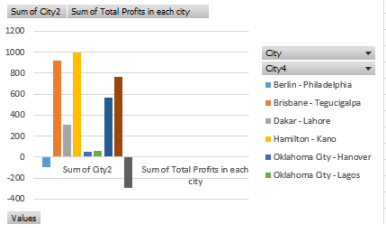
- Drag Product → Rows
- Drag Region → Columns
- Drag Sales → Values (Sum of Sales)



Step 5: Create Pivot Chart

1. Click anywhere on the Pivot Table
2. Go to Insert → PivotChart
3. Click OK

Values	Column Labels ⊟Berlin Philadelphia	Berlin Total	⊟Brisbane Tegucigalpa	Brisbane Total	⊟Dakar Lahore	Dakar Total	⊟Hamilton Kano	Hamilton Total	⊟Oklahoma City Hanover	Lagos	Oklahoma City Total	⊟Porirua Manila	Porirua Total	⊟Sydney Stockholm	Sydney Total	⊟Wollongong Istanbul	Wollongong Total	Grand T
Sum of City2	-$96.54	-$96.54	$919.97	$919.97	######	$311.52	$996.48	$996.48	$54.71	######	$116.87	$564.84	$564.84	$763.28	$763.28	-$288.77	-$288.77	$3,28
Sum of Total Profits in each city	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0



Practical - 2

Aim : Apply the what-if Analysis for data visualization. Design and generate necessary reports based on the data warehouse data. Use Excel

To perform What-If Analysis on data warehouse data using Microsoft Excel and design visual reports to analyze the impact of changing key variables on business outcomes.

Step 1: Prepare the Data Warehouse Data

1. Open Microsoft Excel
2. Create a new workbook
3. Import or enter data with the following structure :

Name	Java	AI	IOT	SPM	AWP	TOTAL	Percetage
Sohel	60	60	54	68	68	310	62
Dinesh	62	61	51	86	48	308	61.6
Fardeen	58	52	48	35	59	252	50.4
Deepak	49	58	68	78	80	333	66.6
Raj	62	36	46	75	69	288	57.6
Pal	89	86	68	56	68	367	73.4

4. Calculate Percentage using formula:

=(Total/500)*100

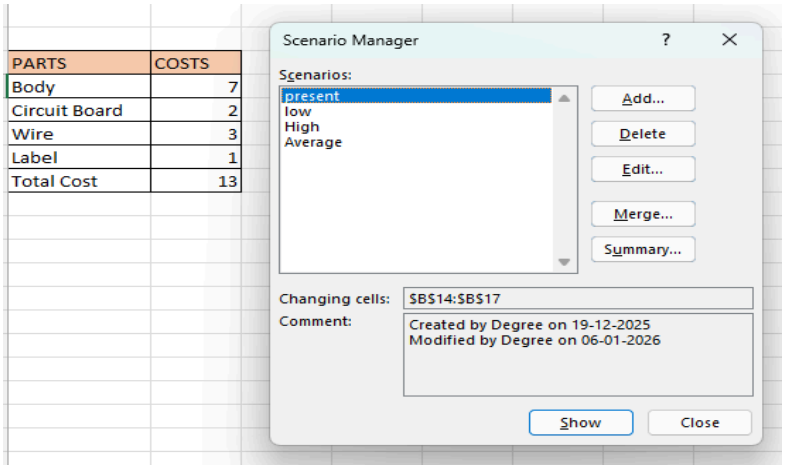
Step 2: Apply What-If Analysis (Scenario Manager)

Scenario 1: Price Increase

1. Go to Data → What-If Analysis → Scenario Manager
2. Click Add
3. Scenario Name: Price Increase
4. Changing Cells: Select Unit Price
5. Enter increased values (e.g., +10%)

Scenario 2: Price Decrease

1. Add another scenario
2. Scenario Name: Price Decrease
3. Reduce Unit Price (e.g., −10%)
4. Click Summary
5. Choose Scenario Summary
6. ce (e.g., −10%)
7. Click **Summary**
8. Choose **Scenario Summary**



Scenario Summary

	Current Values:	present	low	High	Average
Changing Cells:					
B14	7	7	7	7	7
B15	2	2	2	2	2
B16	3	3	3	3	3
B17	1	1	1	1	1
Result Cells:					
B18	13	13	13	13	13

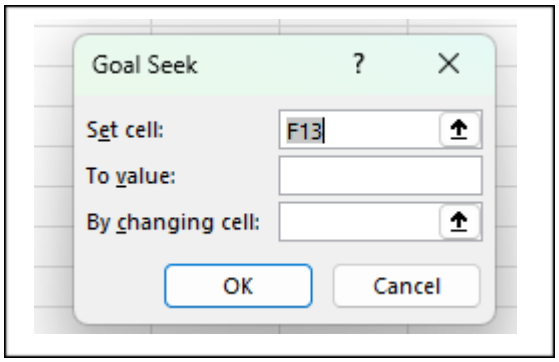
Notes: Current Values column represents values of changing cells at time Scenario Summary Report was created. Changing cells for each scenario are highlighted in gray.

Step 3: Apply What-If Analysis using Data Tables

One-Variable Data Table

1. Create a column with different **Percentage values**
2. Link the percentage formula to the top cell
3. Select the table
4. Go to **Data → What-If Analysis → Data Table**
5. Set:
    - Column Input Cell → Unit Price cell

Total	Percentage
	62
350	70
400	80
412	82.4
312	62.4
150	30
100	20
492	98.4
497	99.4



Step 4: Goal Seek Analysis

Objective: Find Unit Price required to achieve target revenue

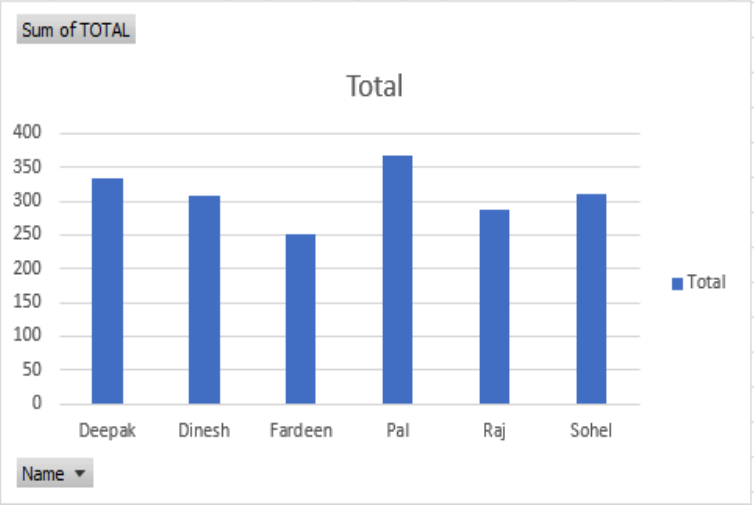
1. Go to **Data → What-If Analysis → Goal Seek**
2. Set Cell → Revenue
3. To Value → 10,000,000
4. By Changing Cell → Unit Price
5. Click Ok

Step 5: Data Visualization (Charts)

1. Select Pivot Table data
2. Go to **Insert → Charts**

Sum of TOTAL	Column Labels						
Row Labels	36	52	58	60	61	86	Grand Total
Deepak			333				333
Dinesh					308		308
Fardeen		252					252
Pal						367	367
Raj	288						288
Sohel				310			310
Grand Total	**288**	**252**	**333**	**310**	**308**	**367**	**1858**

Row Labels	Sum of TOTAL
Deepak	333
Dinesh	308
Fardeen	252
Pal	367
Raj	288
Sohel	310
Grand Total	**1858**



PRACTICAL NO - 03

Aim: Perform tha data classification using classification algorithm using R/python

Python Code:

```
# Data Classification using Random Forest (Iris Dataset)

import pandas as pd
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

# 1. Load Dataset
iris = load_iris()
X = pd.DataFrame(iris.data, columns=iris.feature_names)
y = iris.target

# 2. Split into Training and Testing Data
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)

# 3. Create and Train the Model
model = RandomForestClassifier(n_estimators=100, random_state=42)
model.fit(X_train, y_train)
```

```
# 4. Make Predictions
y_pred = model.predict(X_test)

# 5. Evaluate the Model
print("Accuracy:", accuracy_score(y_test, y_pred))
print("\nClassification Report:\n", classification_report(y_test, y_pred))
print("\nConfusion Matrix:\n", confusion_matrix(y_test, y_pred))
```

Output :

```
Accuracy: 1.0

Classification Report:
               precision    recall  f1-score   support

           0       1.00      1.00      1.00        19
           1       1.00      1.00      1.00        13
           2       1.00      1.00      1.00        13

    accuracy                           1.00        45
   macro avg       1.00      1.00      1.00        45
weighted avg       1.00      1.00      1.00        45

Confusion Matrix:
 [[19  0  0]
 [ 0 13  0]
 [ 0  0 13]]
```

PRACTICAL NO - 04

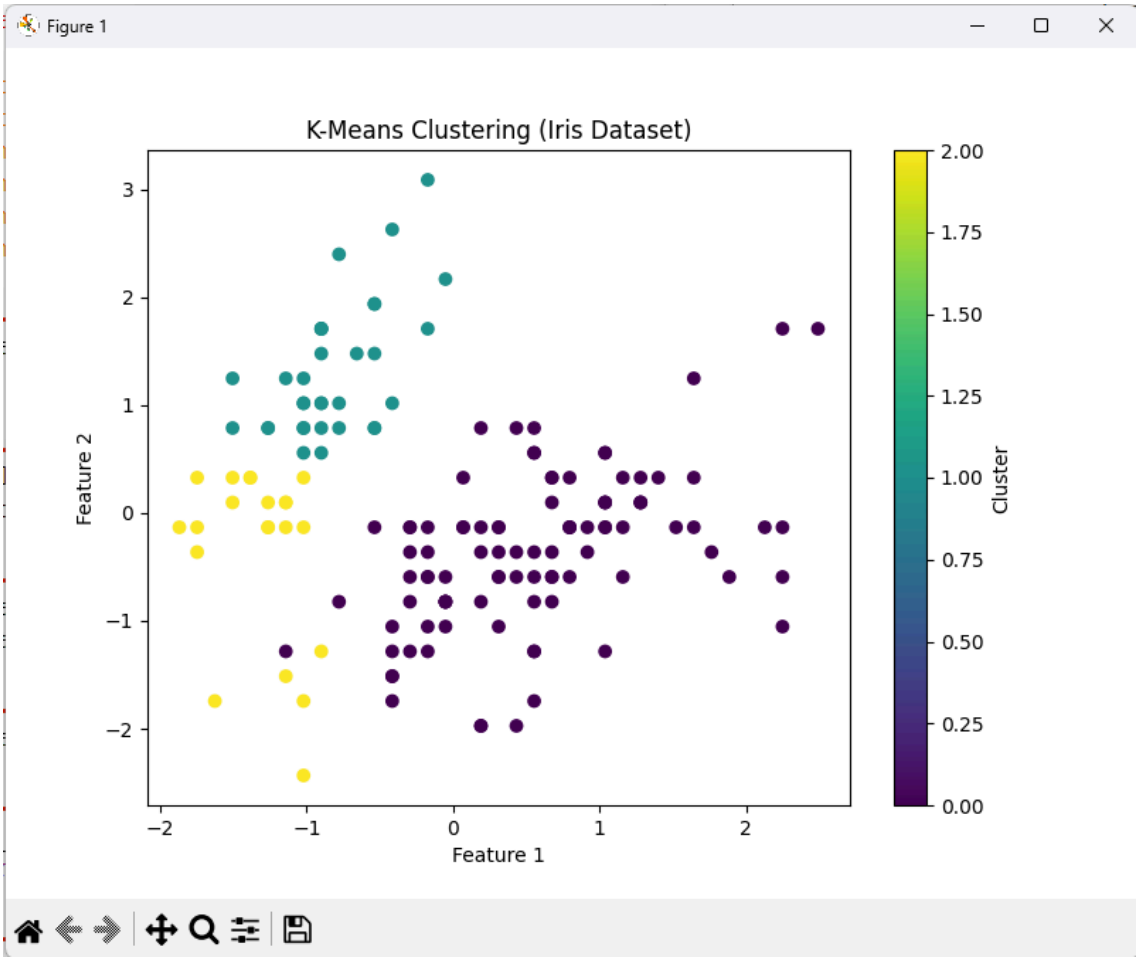
Aim: Perform the data clustering using clustering algorithm using R/python

Python Code:

```
# Data Clustering using K-Means (Iris Dataset)

import pandas as pd
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import silhouette_score

# 1. Load Dataset
iris = load_iris()
X = pd.DataFrame(iris.data, columns=iris.feature_names)

# 2. Feature Scaling (important for clustering)
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# 3. Apply K-Means Clustering
kmeans = KMeans(n_clusters=3, random_state=42)
kmeans.fit(X_scaled)

# 4. Get Cluster Labels
```

```
clusters = kmeans.labels_

# 5. Evaluate Clustering
sil_score = silhouette_score(X_scaled, clusters)
print("Silhouette Score:", sil_score)

# 6. Add Clusters to DataFrame
X['Cluster'] = clusters
print(X.head())

# 7. Visualize Clusters (2D using first two features)
plt.figure(figsize=(8,6))
plt.scatter(X_scaled[:, 0], X_scaled[:, 1], c=clusters, cmap='viridis')
plt.title("K-Means Clustering (Iris Dataset)")
plt.xlabel("Feature 1")
plt.ylabel("Feature 2")
plt.colorbar(label='Cluster')
plt.show()
```

Output:



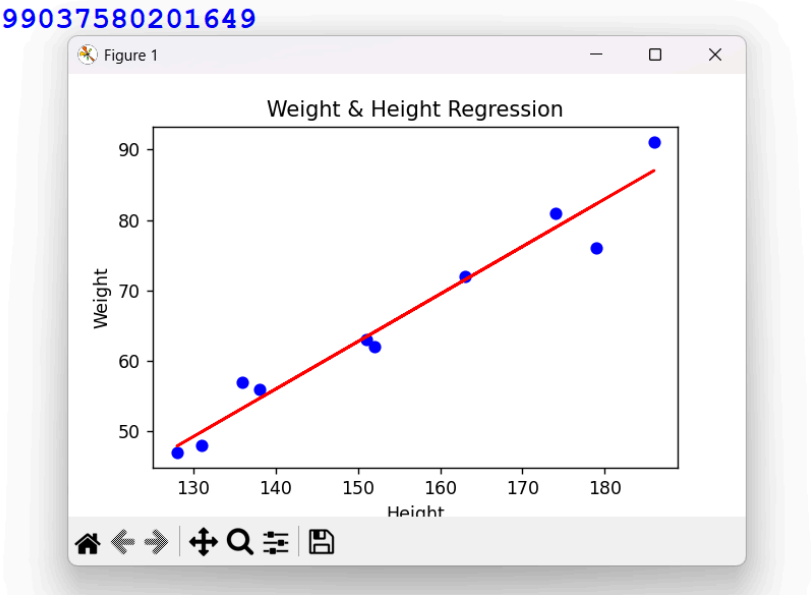

```
Silhouette Score: 0.4798814508199817
   sepal length (cm)  sepal width (cm)  ...  petal width (cm)  Cluster
0                5.1               3.5  ...               0.2        1
1                4.9               3.0  ...               0.2        2
2                4.7               3.2  ...               0.2        2
3                4.6               3.1  ...               0.2        2
4                5.0               3.6  ...               0.2        1

[5 rows x 5 columns]
```

PRACTICAL NO. 5

```
#Import required libraries
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression

#Create data
x=np.array([151, 174, 138, 186, 128, 136, 179, 163, 152,
131]).reshape(-1, 1)
y=np.array([63, 81, 56, 91, 47, 57, 76, 72, 62, 48])

#Build linear regression model
model=LinearRegression()
model.fit(x, y)

#Predict value for x = 140
prediction=model.predict([[140]])
print("Predicted value for x=140:", prediction[0])

#Print model parameters
print("Intercept:", model.intercept_)
```

```
print("Slope:", model.coef_[0])



#Plot graph

plt.scatter(x, y, color='blue')
plt.plot(x, model.predict(x), color='red')
plt.title("Weight & Height Regression")
plt.xlabel("Height")
plt.ylabel("Weight")

plt.savefig("linearreg.png")
plt.show()
```

```
========================= RESTART: C:/python/pract5.py ====================
Predicted value for x=140: 55.99037580201649
Intercept: -38.45508707607701
Slope: 0.6746104491292393
```

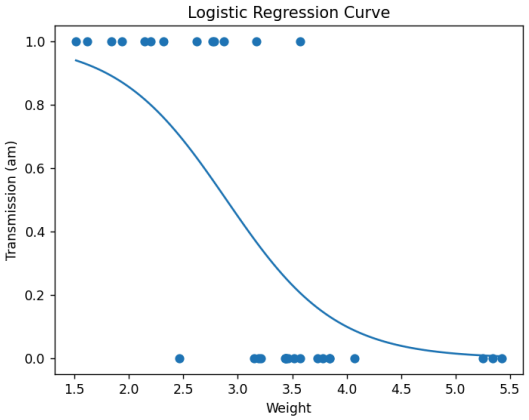


PRACTICAL NO 6

```
# Import required libraries
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd

from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report
from pydataset import data

# Load dataset
mtcars = data('mtcars')

# Select required columns
input_data = mtcars[['am', 'cyl', 'hp', 'wt']]
print(input_data.head())

# Define independent and dependent variables
X = input_data[['cyl', 'hp', 'wt']]
y = input_data['am']

# Build logistic regression model
log_model = LogisticRegression()
log_model.fit(X, y)
```

```
# Print coefficients
print("Intercept:", log_model.intercept_)
print("Coefficients:", log_model.coef_)

# Predict
y_pred = log_model.predict(X)

# Classification report
print(classification_report(y, y_pred))

# Plot using wt only
X1 = input_data[['wt']]
model2 = LogisticRegression()
model2.fit(X1, y)

plt.scatter(X1, y)

x_test = np.linspace(X1.min(), X1.max(), 100).reshape(-1, 1)
plt.plot(x_test, model2.predict_proba(x_test)[:, 1])

plt.xlabel("Weight")
plt.ylabel("Transmission (am)")
plt.title("Logistic Regression Curve")
plt.show()
```

```
                   am  cyl   hp     wt
Mazda RX4           1    6  110  2.620
Mazda RX4 Wag       1    6  110  2.875
Datsun 710          1    4   93  2.320
Hornet 4 Drive      0    6  110  3.215
Hornet Sportabout   0    8  175  3.440
Intercept: [6.97297795]
Coefficients: [[-0.71109921  0.02113021 -1.9902589 ]]
              precision    recall  f1-score   support

           0       0.90      0.95      0.92        19
           1       0.92      0.85      0.88        13

    accuracy                           0.91        32
   macro avg       0.91      0.90      0.90        32
weighted avg       0.91      0.91      0.91        32
```



Practical - 7

Aim : Write python program to read from a CSV file, perform simple data analysis and generate basic insights.(Use Pandas is a Python library)

Step 1: Create table in excel, save as "Sales.csv"

Category	Sales
BMW	4000
AUDI	5617
Ferrari	4575
Lamborgh	5545
Mercedes	7854
Porshe	5241
Dodge	6542
Maserati	8521
Bentley	1252
Aston Mar	5563
Ford Must	9542
Nissan	2552

Step 2 : if pandas module not found:

pip install pandas

Step 3: Code Saved as Practical7.py

```
import pandas as pd

# Read CSV file

data = pd.read_csv("sales.csv")

print("Data from CSV file:")

print(data)

# Basic information

print("\nBasic Information:")

print(data.info())

# Statistical summary

print("\nStatistical Summary:")

print(data.describe())
```

```
# Calculated insights (from the same table)
total_sales = data["Sales"].sum()
average_sales = data["Sales"].mean()
highest_sales = data["Sales"].max()
lowest_sales = data["Sales"].min()
print("\nInsights:")
print("Total Sales:", total_sales)
print("Average Sales:", average_sales)
print("Highest Sales:", highest_sales)
print("Lowest Sales:", lowest_sales)
# Category-wise sales (still same table, just grouped)
category_sales = data.groupby("Category")["Sales"].sum()
print("\nCategory-wise Sales:")
print(category_sales)
```

Output

```
========================= RESTART: D:/deepak/Bi/7a.py
Data from CSV file:
        Category  Sales
0            BMW   4000
1           AUDI   5617
2        Ferrari   4575
3    Lamborghini   5545
4  Mercedes-Benz   7854
5         Porshe   5241
6          Dodge   6542
7       Maserati   8521
8        Bentley   1252
9   Aston Martin   5563
10  Ford Mustang   9542
11         Nissan   2552

Basic Information:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 12 entries, 0 to 11
Data columns (total 2 columns):
 #   Column    Non-Null Count  Dtype
---  ------    --------------  -----
 0   Category  12 non-null     object
 1   Sales     12 non-null     int64
dtypes: int64(1), object(1)
memory usage: 324.0+ bytes
None
```

```
Statistical Summary:
              Sales
count     12.000000
mean    5567.000000
std     2373.451917
min     1252.000000
25%     4431.250000
50%     5554.000000
75%     6870.000000
max     9542.000000

Insights:
Total Sales: 66804
Average Sales: 5567.0
Highest Sales: 9542
Lowest Sales: 1252

Category-wise Sales:
Category
AUDI                5617
Aston Martin        5563
BMW                 4000
Bentley             1252
Dodge               6542
Ferrari             4575
Ford Mustang        9542
Lamborghini         5545
Maserati            8521
Mercedes-Benz       7854
Nissan              2552
Porshe              5241
Name: Sales, dtype: int64
```

Practical - 8(a)

Aim : Perform data visualization using Python on any sales data.

Step 1: Install Required Libraries if pandas module not found:

→pip install pandas

Step 2 : Import Libraries

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

→pandas (pd) → Used to create and manage data in table format (DataFrame).

→matplotlib.pyplot (plt) → Used for creating graphs and charts.

→seaborn (sns) → Used for advanced and attractive statistical visualizations.

Step 3 : Create or Load Sales Data

```
data = {
    "years": ["Jan", "Feb", "Mar", "Apr", "May", "Jun"],
    "Product_A": [12000, 15000, 17000, 16000, 18000, 20000],
    "Product_B": [10000, 12000, 14000, 13000, 15000, 17000],
    "Product_C": [8000, 9000, 10000, 11000, 12000, 14000]
}
```

→ A dictionary is created.

"years" column contains months (Jan–Jun).

"Product_A", "Product_B", "Product_C" contain monthly sales values.

Step 4: Code Saved as Practical7.py

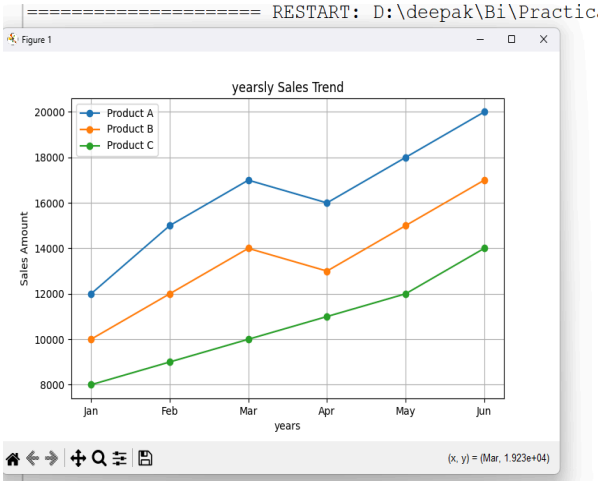
```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns


# ----------------------------
# Create Sample Sales Data
# ----------------------------
data = {
    "years": ["Jan", "Feb", "Mar", "Apr", "May", "Jun"],
    "Product_A": [12000, 15000, 17000, 16000, 18000, 20000],
```

```
    "Product_B": [10000, 12000, 14000, 13000, 15000, 17000],
    "Product_C": [8000, 9000, 10000, 11000, 12000, 14000]
}

df = pd.DataFrame(data)

# ----------------------------
# Line Plot – yearsly Sales
# ----------------------------
plt.figure(figsize=(8, 5))
plt.plot(df["years"], df["Product_A"], marker='o', label="Product A")
plt.plot(df["years"], df["Product_B"], marker='o', label="Product B")
plt.plot(df["years"], df["Product_C"], marker='o', label="Product C")

plt.title("yearsly Sales Trend")
plt.xlabel("years")
plt.ylabel("Sales Amount")
plt.legend()
plt.grid(True)
plt.show()

# ----------------------------
# Bar Chart – Sales Comparison
# ----------------------------
df.set_index("years").plot(kind="bar", figsize=(8, 5))
plt.title("Sales Comparison by Product")
plt.xlabel("years")
plt.ylabel("Sales Amount")
```

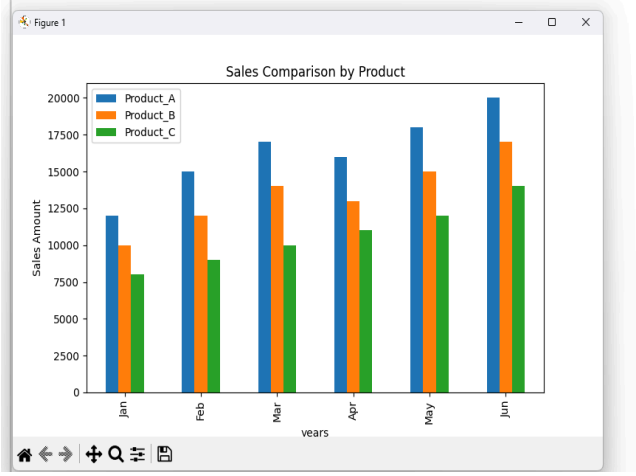
```
plt.show()
# ----------------------------
# Pie Chart – Total Sales Share
# ----------------------------
total_sales = df[["Product_A", "Product_B", "Product_C"]].sum()

plt.figure(figsize=(6, 6))
plt.pie(total_sales, labels=total_sales.index, autopct='%1.1f%%')
plt.title("Total Sales Distribution")
plt.show()

# ----------------------------
# Heatmap – Sales Intensity
# ----------------------------
plt.figure(figsize=(8, 5))
sns.heatmap(df.drop("years", axis=1), annot=True, cmap="YlGnBu")
plt.title("Sales Heatmap")
plt.show()
```

Output

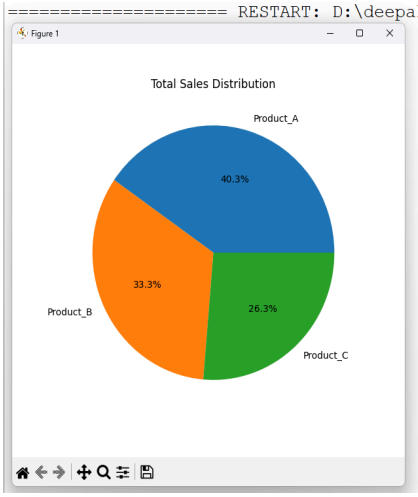
```
===================== RESTART: D:\deepak\Bi\Practical-8a.py
```




```
===================== RESTART: D:\deepak\Bi\Practical-8a.py
```




```
===================== RESTART: D:\deepak\Bi\Practical-8a.py
```




```
===================== RESTART: D:\deepak\Bi\Practical-8a.py
```

Practical – 8(b)

Aim : Perform data visualization using PowerBI on any sales data.

Step 1: Install Required Libraries if pandas module not found:

→ Open PowerBI Desktop

→ Click Get Data → Excel/CSV

→ Select sales dataset

→ Click Load

Step 2: Line Chart-Product Wise Sales

•Axis → Product

•Values → Sales

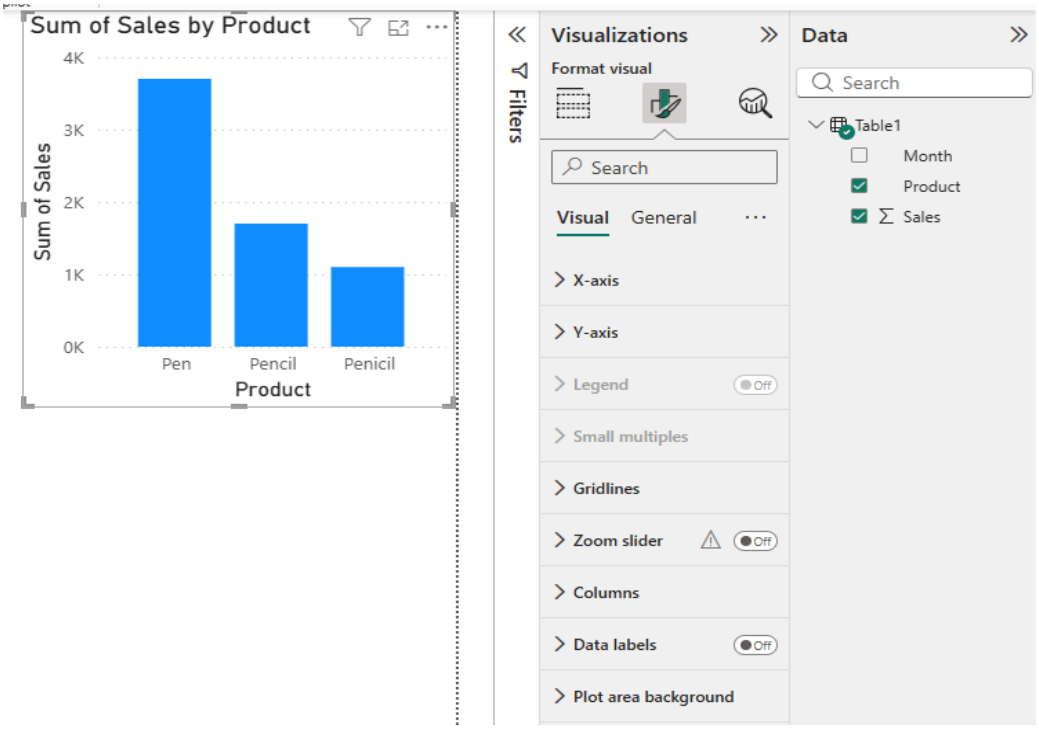
Step 3: Donut Chart – Sales by Month

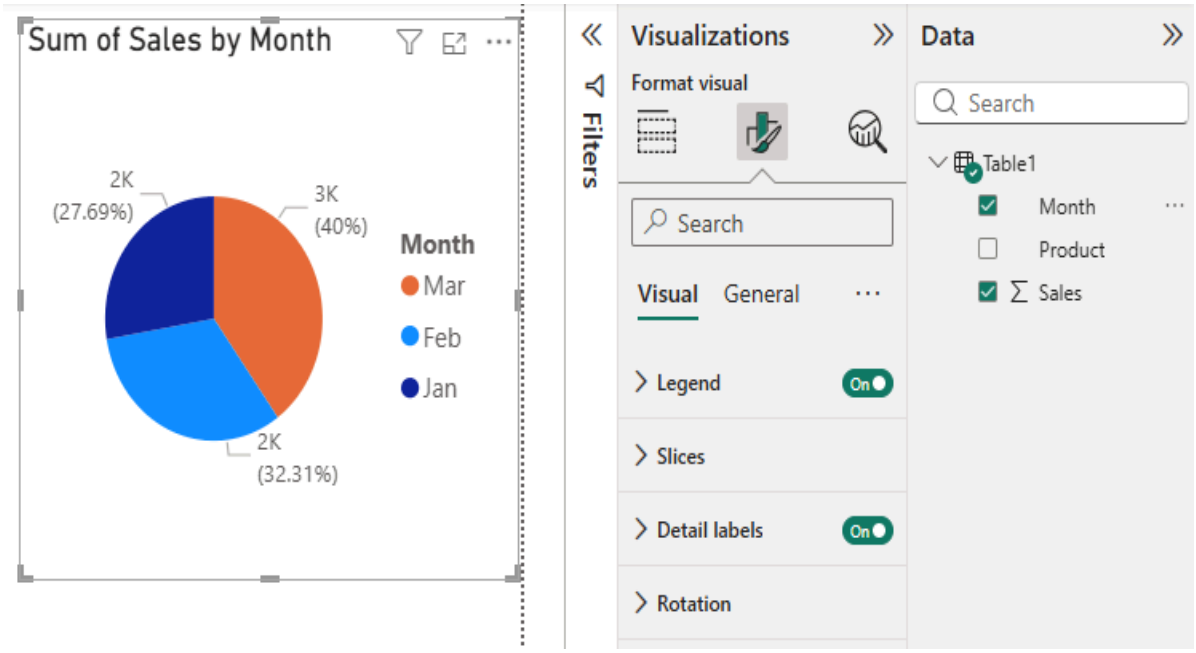
•Legend → Month

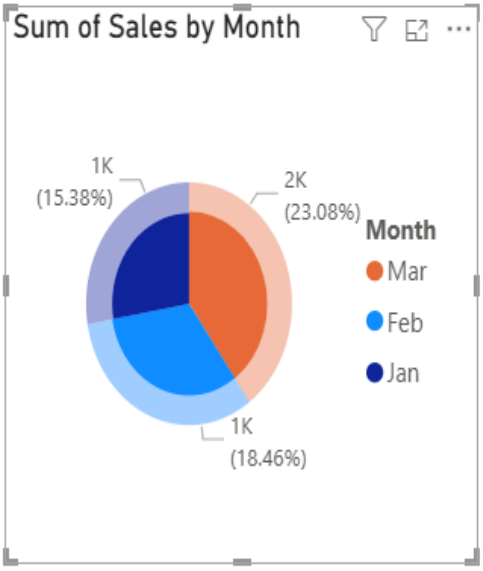
•Values → Sales

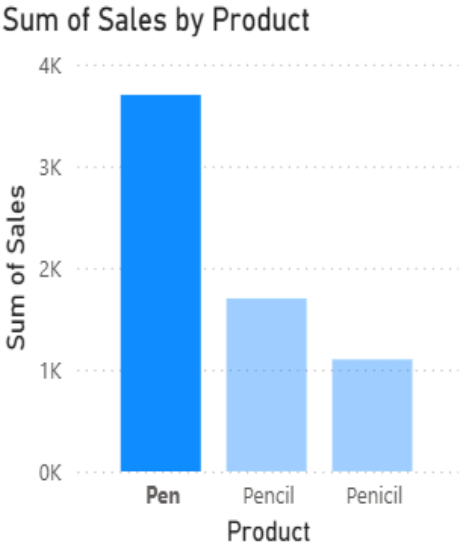
Step 4: Displays contribution of each category

Output :









Result:

- Sales data was successfully analyzed and visualized using Microsoft Power BI.
- An interactive dashboard was created to evaluate business performance effectively

PRACTICAL NO. 09

Aim: Create a Data Staging area for the selected database using SQL.

STAGING AREA

It is the storage for the sources data used before the ETL process. The source data is preloaded into the staging area which is a separate database with a similar table structure to that of the original data source.

PREPARING THE STAGING AREA FOR THE DATA WAREHOUSE

1. Download or obtain the source data set in **CSV** format.
2. If the dataset is in the form of a denormalized table you can remove any unwanted columns or rows if applicable. It will speed up the load and also simplify the ETL process later.
3. Launch the **SQL Server Management Studio** and connect to the desired database engine. Make sure you have an account with privileges to create databases when connecting.
4. Navigate to the **Object Explorer** and select **Databases**, right click and Select New Database...and create the database as required to store the staging data.
5. Select the newly created database, right click and select **New Query** and create the required table schema to load the source data.

To create a single staging table called county_stage using the below script and execute it by pressing F5 key. create table county_stage(id int state varchar (30), countyname varchar(100));

1. After the staging area database was created the loading of the data needs to be done. For this the Import function of the SQL Server Management Studio was used. Following are the steps.
2. Right click on the created staging database, select **Tasks** and click on **Import Data… Click on Next.**

In Choose data source as **Flat File Source** and browse your **CSV file** and check the columns in

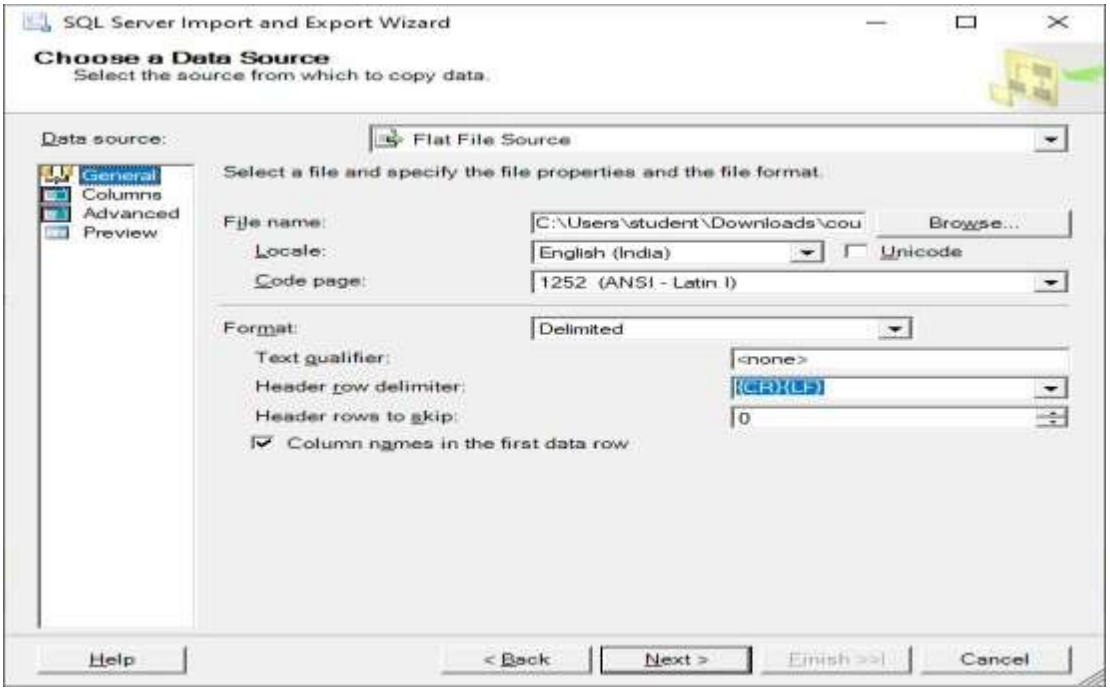
first data row.

Then click on **Columns tab** and set the proper delimiters and check the columns tab to view if the configuration is working if not set the proper delimiters to get the desired column and row organization.

Next move to **Advanced Tab** and configure properties of each column such as for **ID** set data type as <u>four byte signed integer,</u> **State** set data type as <u>string</u> and OutputColumn Width as 30 and for **County Name** type as string and OutputColumn Width as 100.

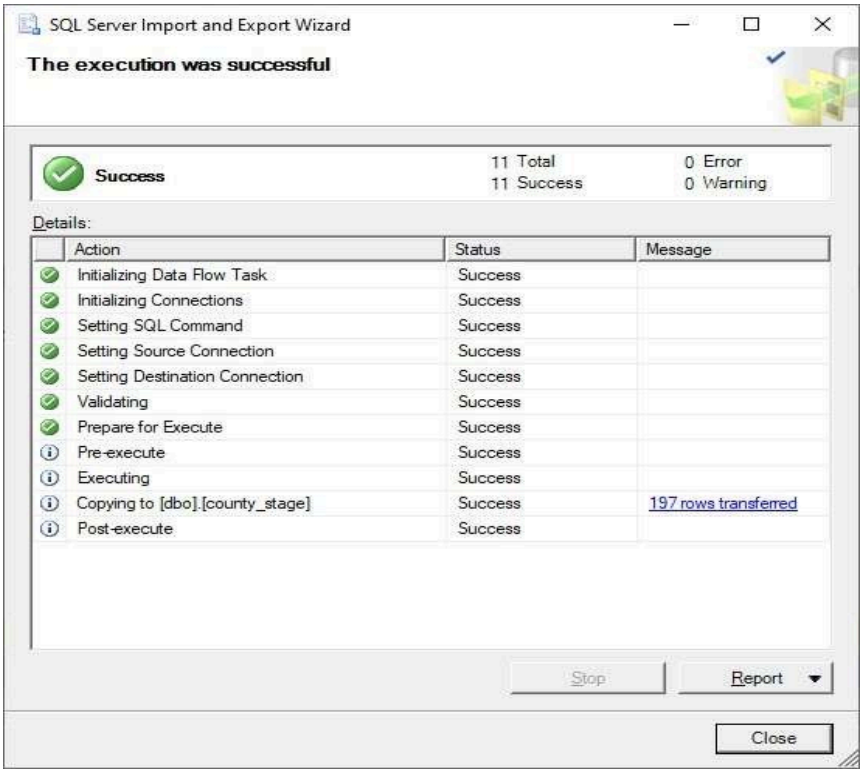
Move to the **Preview** tab to get a limited preview of the data to be important under given columns, Click Next.

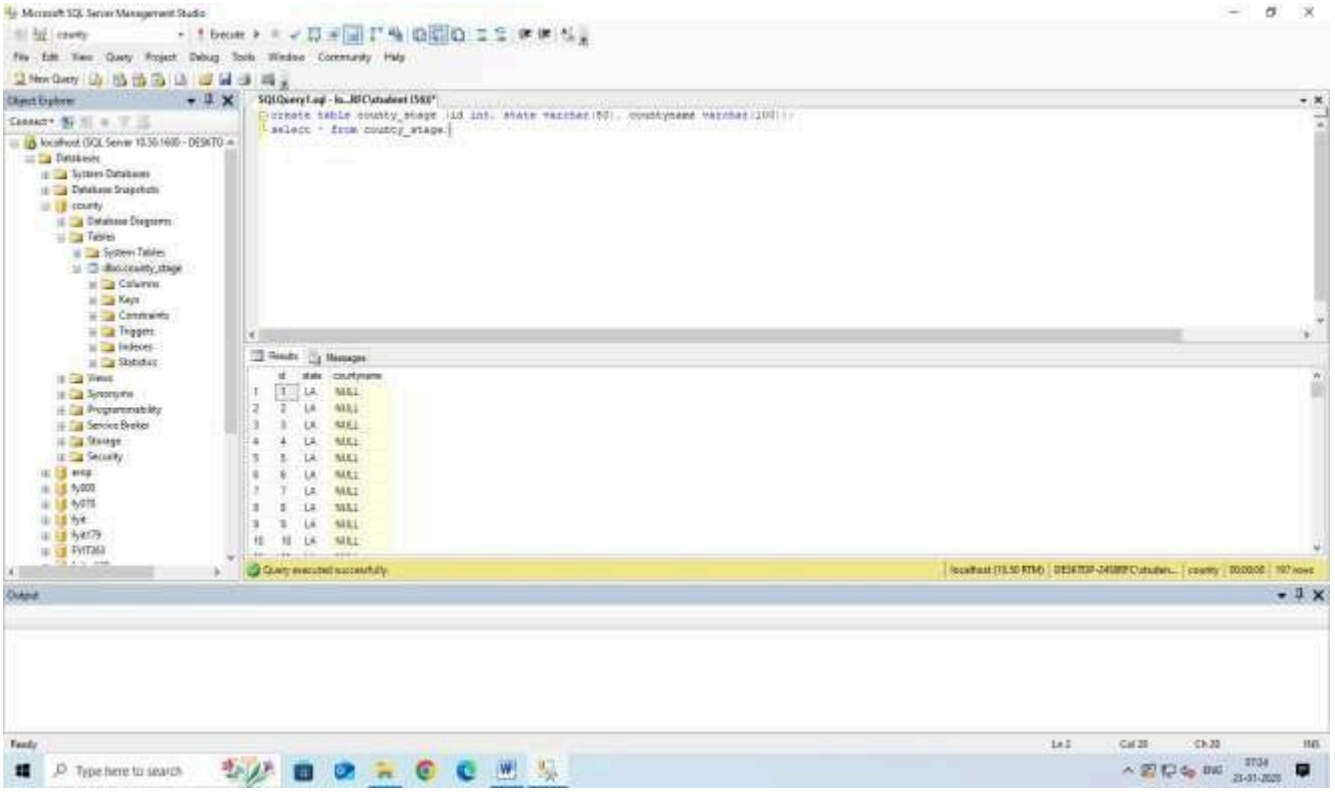
On the Choose a **Destination** page select appropriate **Destination Server Name(localhost)** and provide a require authentication to connect (if required) and you will be able to select the staging database created above the destination Database(**County**) Click **Next**.



On the **Select Source Tables** and Views page, map the source to the destination table(s) as required. Click **Next.**

On the **Save** and **Run Package** page select **Run Immediately** to import the data after configuring the wizard. If you need to connect with a SSIS package you can configure it here. Click Next.


SQL Server Import and Export Wizard
The execution was successful
Success
11 Total
11 Success
0 Error
0 Warning
Details:
Action
Status
Message
Initializing Data Flow Task
Success
Initializing Connections
Success
Setting SQL Command
Success
Setting Source Connection
Success
Setting Destination Connection
Success
Validating
Success
Prepare for Execute
Success
Pre-execute
Success
Executing
Success
Copying to [dbo].[county_stage]
Success
197 rows transferred
Post-execute
Success
Stop
Report
Close



PRACTICAL NO. 10

Aim: Create Cube with suitable dimensions and fact tables based on ROLAP, MOLAP and HOLAP model.

<u>Creating Data Warehouse</u>

Follow the given steps to run the query in SSMS (SQL Server Management Studio)

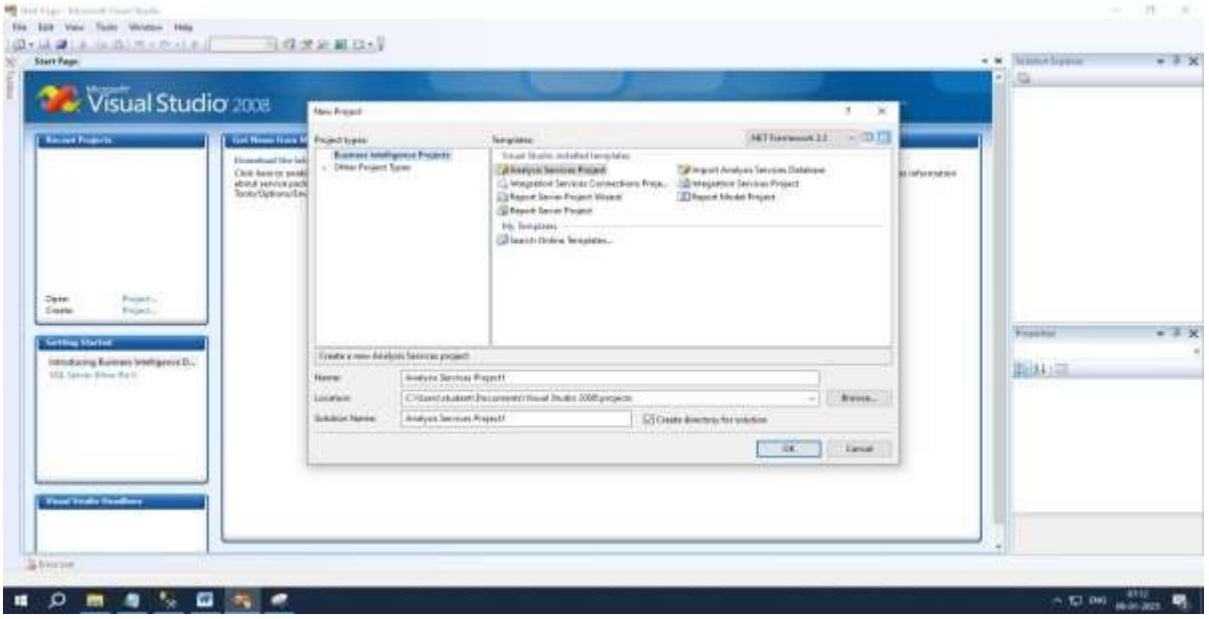
1. Open SQL Server Management Studio 2008
2. Connect Database Engine
3. Open New Query Editor
4. Open SQL Script in query editor
5. To run the given SQL Script, press F5
6. It will create and populate “Sales_DW” database on your SQL Server

<u>Developing an OLAP Cube</u>

For creation of OLAP Cube in Microsoft BIDS Environment, follow the 10 easy steps given below.

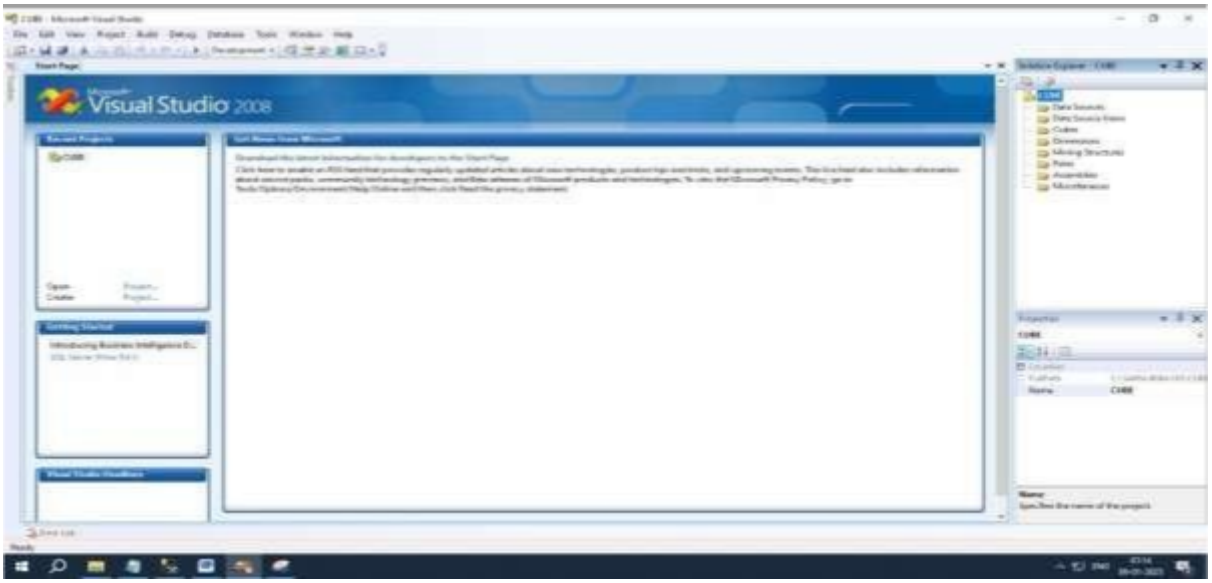
Step 1: Start BIDS Environment. Click on Start ->Microsoft SQL Server 2008 R2 -> Click SQL Server Business Intelligence Development Studio.

Step 2: Start *Analysis Services Project*. Click File -> New -> Project -> Business Intelligence Projects -> select Analysis Services Project -> Assign Project Name (cubepract4) ->Click OK.



Step 3: Creating New Data Source

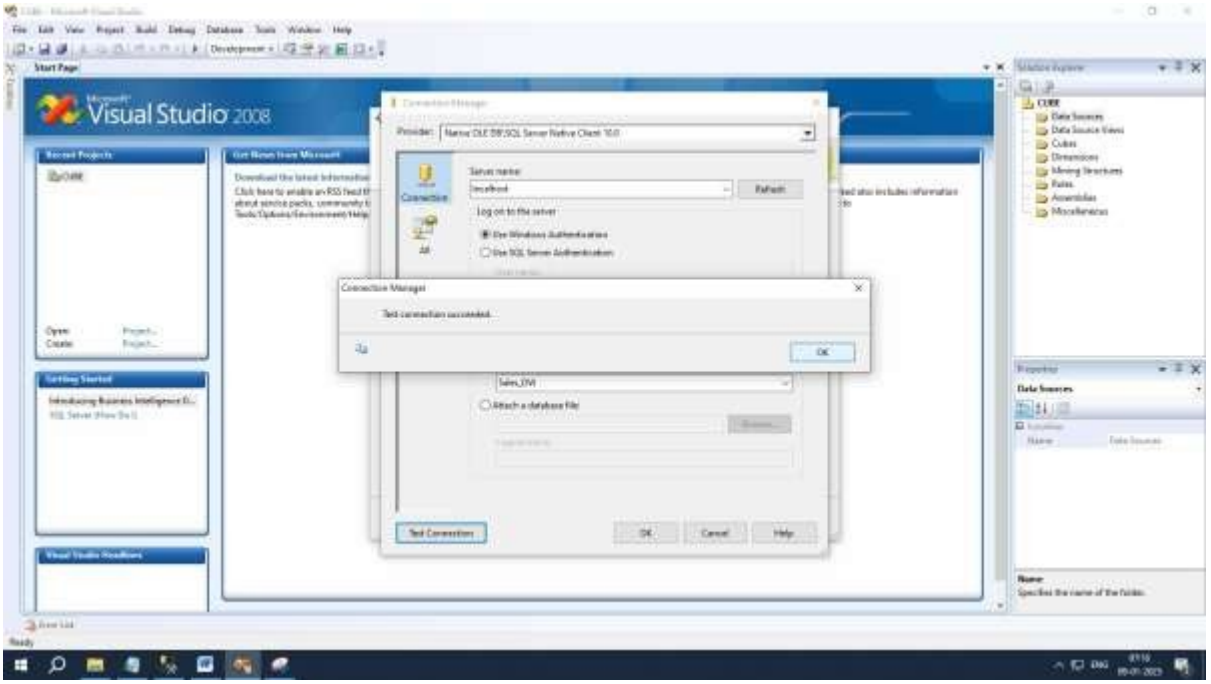
In Solution Explorer, Right click on Data Source -> Click New Data Source



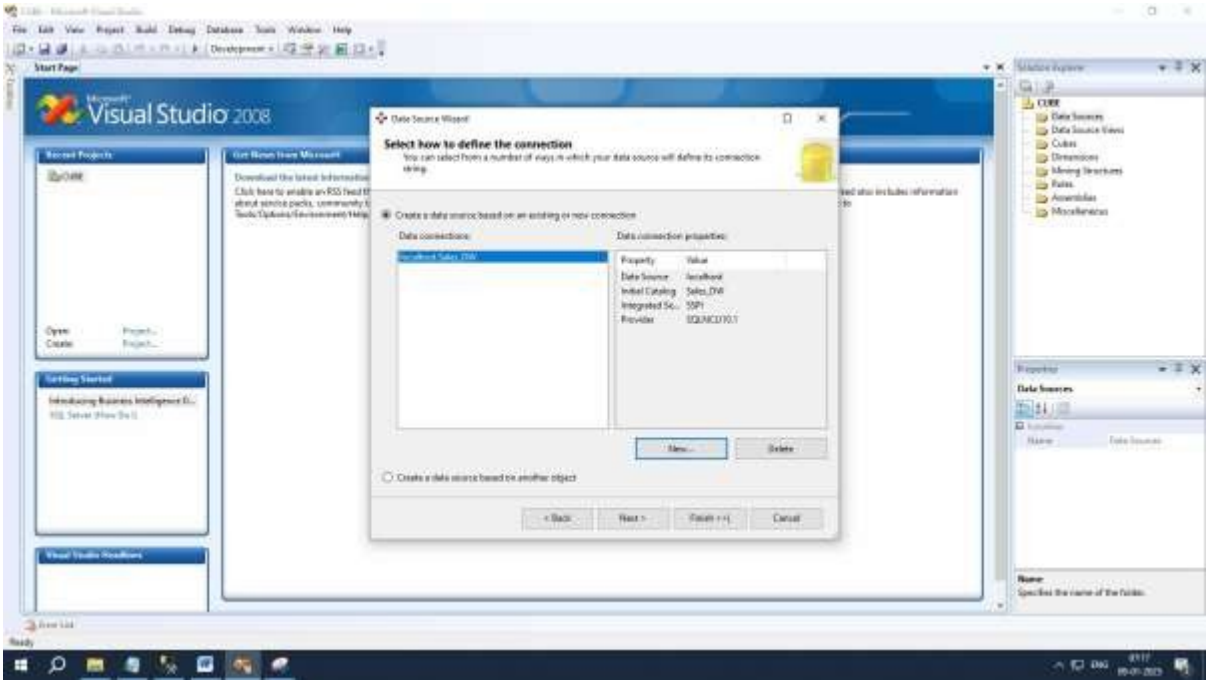
3.1 Click on Next

3.2 Click on New Button Creating New connection

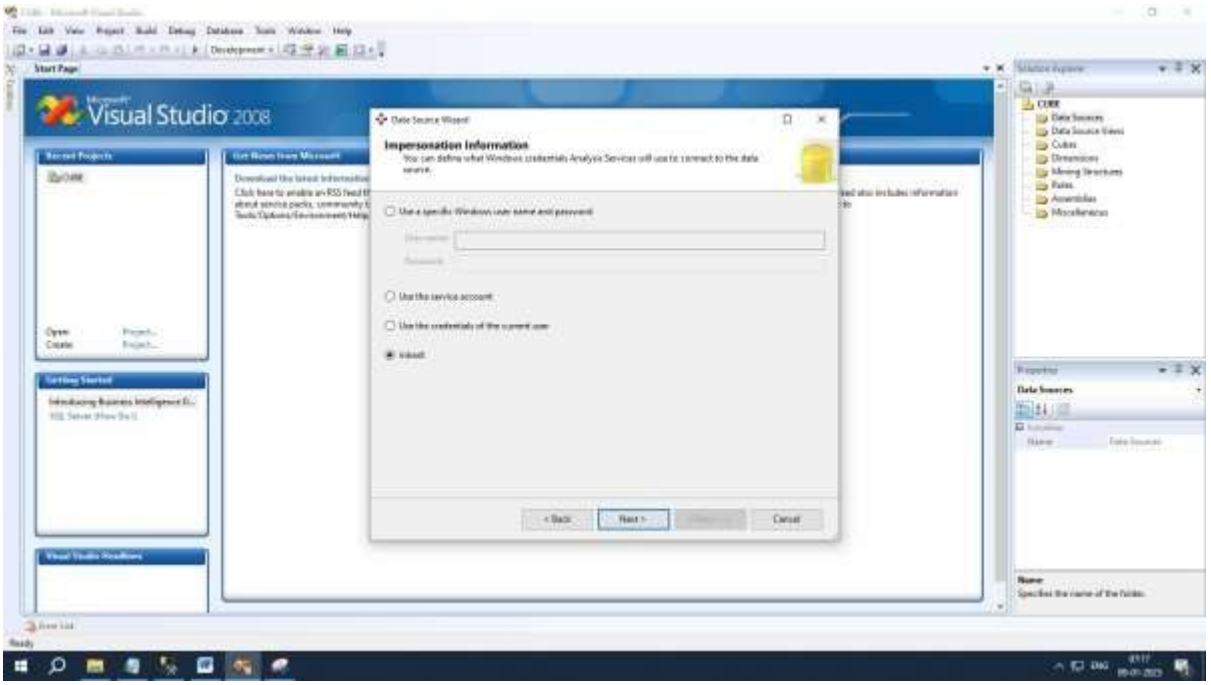
1. Specify your SQL Server Name where your Data Warehouse was created
2. Select Radio Button according to your SQL Server Authentication mode
3. Specify your Credentials using which you can connect to your SQL Server
4. Select database Sales_DW.
5. Click on Test Connection and verify for its success. 6. Click OK



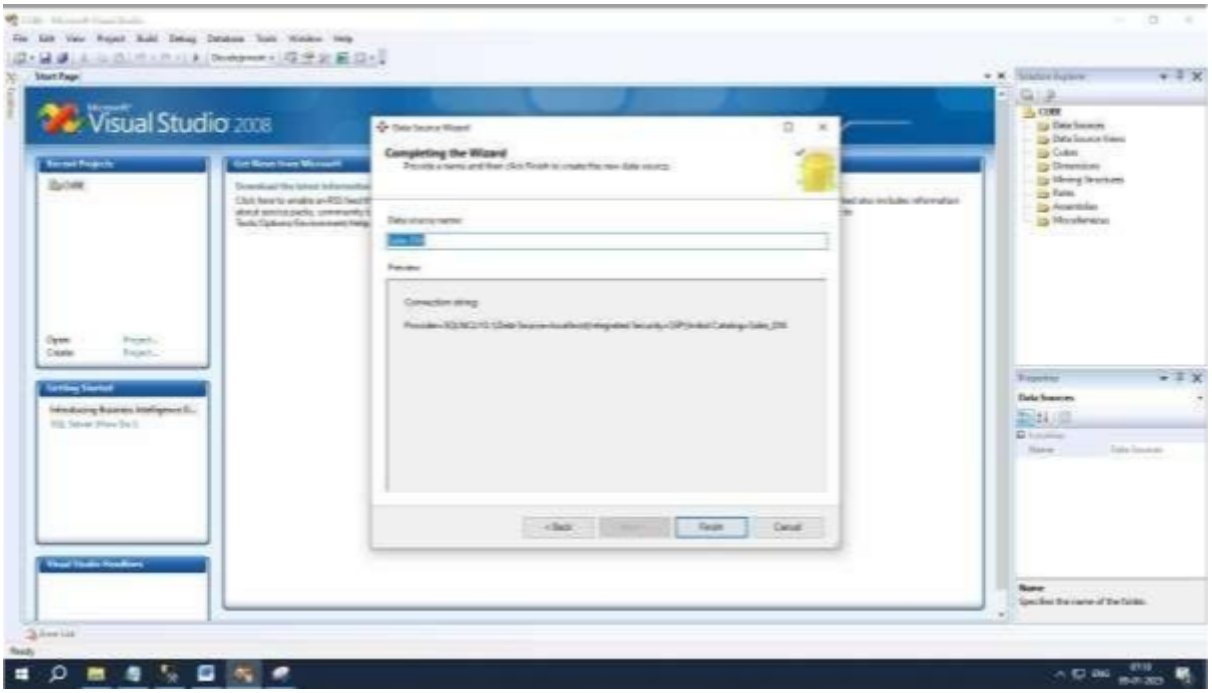
Select Connection created in Data Connections -> Click Next



- Select option Inherit



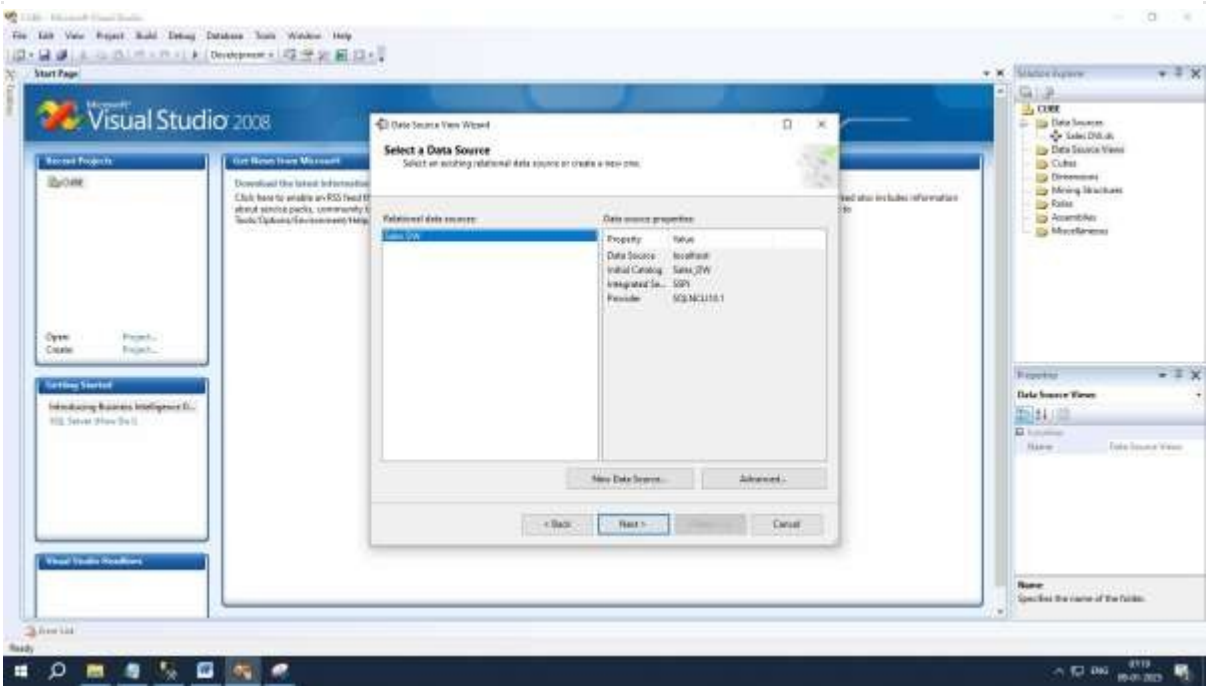
- Assign Data Source Name -> Click Finish



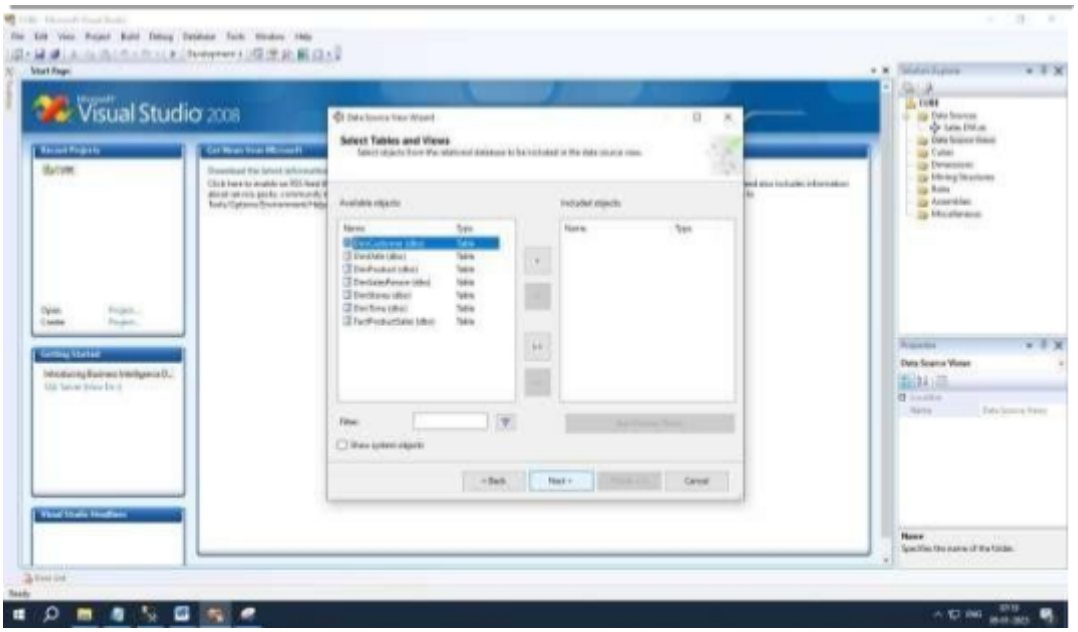
Step 4: Creating New Data Source View

1. In the Solution Explorer, Right Click on Data Source View -> Click on New Data Source View
2. Click Next
3. Select Relational Data Source we have created previously (Sales_DW) -> Click

Next

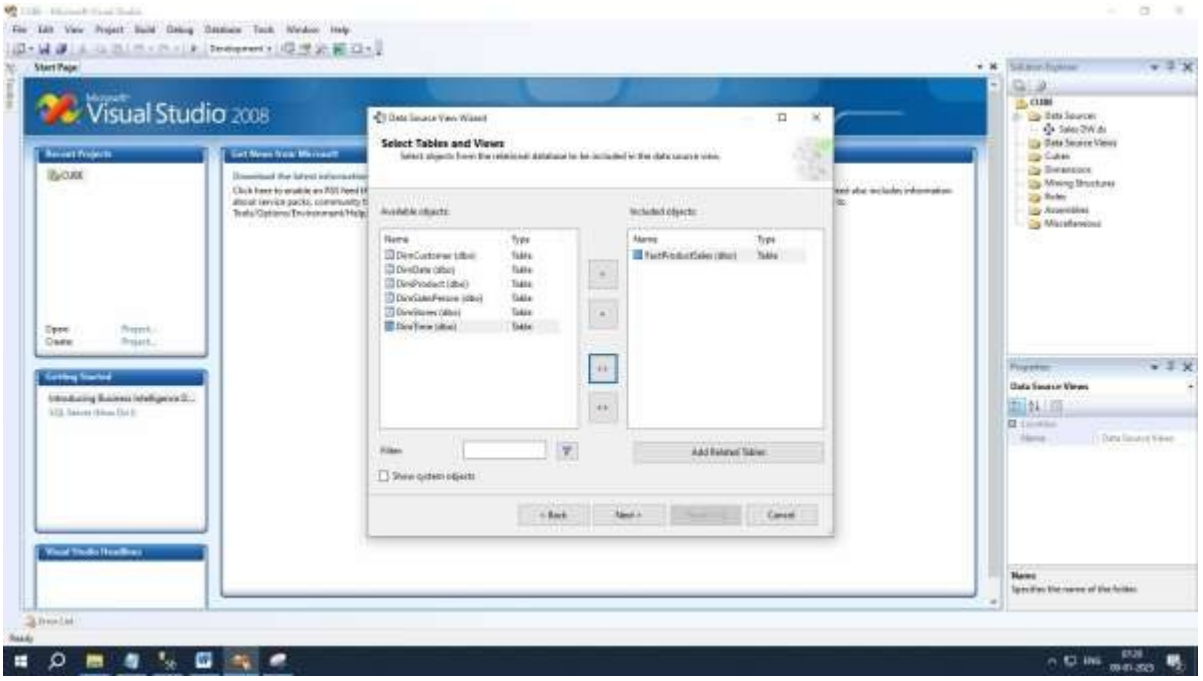


4.First move your Fact Table to the right side to include in object lis



Select FactProductSales Table ->Click on Arrow Button to move the selected object to Right pane.

1. Now to add dimensions which are related to your Fact Table, follow the given steps:


Visual Studio 2008
Select Tables and Views
Available objects:
Included objects:
Add Related Tables
Show system objects
< Back
Next >
Cancel